

Vector Semantics & Embeddings

Word2vec

Sparse versus dense vectors

tf-idf (or PMI) vectors are

- **long** (length $|V| = 20,000$ to $50,000$)
- **sparse** (most elements are zero)

Alternative: learn vectors which are

- **short** (length 50-1000)
- **dense** (most elements are non-zero)

Sparse versus dense vectors

Why dense vectors?

- Short vectors may be easier to use as **features** in machine learning (fewer weights to tune)
- Dense vectors may **generalize** better than explicit counts
- Dense vectors may do better at capturing synonymy:
 - *car* and *automobile* are synonyms; but are distinct dimensions
 - a word with *car* as a neighbor and a word with *automobile* as a neighbor should be similar, but aren't
- **In practice, they work better**

Common methods for getting short dense vectors

“Neural Language Model”-inspired models

- Word2vec (skipgram, CBOW), GloVe

Singular Value Decomposition (SVD)

- A special case of this is called LSA – Latent Semantic Analysis

Alternative to these "static embeddings":

- Contextual Embeddings (ELMo, BERT)
- Compute distinct embeddings for a word in its context
- Separate embeddings for each token of a word

Simple static embeddings you can download!

Word2vec (Mikolov et al)

<https://code.google.com/archive/p/word2vec/>

GloVe (Pennington, Socher, Manning)

<http://nlp.stanford.edu/projects/glove/>

Word2vec

Popular embedding method

Very fast to train

Code available on the web

Idea: **predict** rather than **count**

Word2vec provides various options. We'll do:

skip-gram with negative sampling (SGNS)

Word2vec

Instead of **counting** how often each word w occurs near "*apricot*"

- Train a classifier on a binary **prediction** task:
 - Is w likely to show up near "*apricot*"?

We don't actually care about this task

- But we'll take the learned classifier weights as the word embeddings

Big idea: **self-supervision**:

- A word c that occurs near *apricot* in the corpus counts as the gold "correct answer" for supervised learning
- No need for human labels
- Bengio et al. (2003); Collobert et al. (2011)

Approach: predict if candidate word c is a "neighbor"

1. Treat the target word t and a neighboring context word c as **positive examples**.
2. Randomly sample other words in the lexicon to get negative examples
3. Use logistic regression to train a classifier to distinguish those two cases
4. Use the learned weights as the embeddings

Skip-Gram Training Data

Assume a ± 2 word window, given training sentence:

...lemon, a [tablespoon of apricot jam, a] pinch...

c1 c2 [target] c3 c4

Skip-Gram Classifier

(assuming a +/- 2 word window)

...lemon, a [tablespoon of apricot jam, a] pinch...

c1 c2 [target] c3 c4

Goal: train a classifier that is given a candidate (**w**ord, **c**ontext) pair

(apricot, jam)

(apricot, aardvark)

...

And assigns each pair a probability:

$$P(+ | w, c)$$

$$P(- | w, c) = 1 - P(+ | w, c)$$

Similarity is computed from dot product

Remember: two vectors are similar if they have a high dot product

- Cosine is just a normalized dot product

So:

- $\text{Similarity}(w, c) \propto w \cdot c$

We'll need to normalize to get a probability

- (cosine isn't a probability either)

Turning dot products into probabilities

$$\text{Sim}(w, c) \approx w \cdot c$$

To turn this into a probability

We'll use the sigmoid from logistic regression:

$$P(+|w, c) = \sigma(c \cdot w) = \frac{1}{1 + \exp(-c \cdot w)}$$

$$\begin{aligned} P(-|w, c) &= 1 - P(+|w, c) \\ &= \sigma(-c \cdot w) = \frac{1}{1 + \exp(c \cdot w)} \end{aligned}$$

How Skip-Gram Classifier computes $P(+|w, c)$

$$P(+|w, c) = \sigma(c \cdot w) = \frac{1}{1 + \exp(-c \cdot w)}$$

This is for one context word, but we have lots of context words.
We'll assume independence and just multiply them:

$$P(+|w, c_{1:L}) = \prod_{i=1}^L \sigma(c_i \cdot w)$$

$$\log P(+|w, c_{1:L}) = \sum_{i=1}^L \log \sigma(c_i \cdot w)$$

Skip-gram classifier: summary

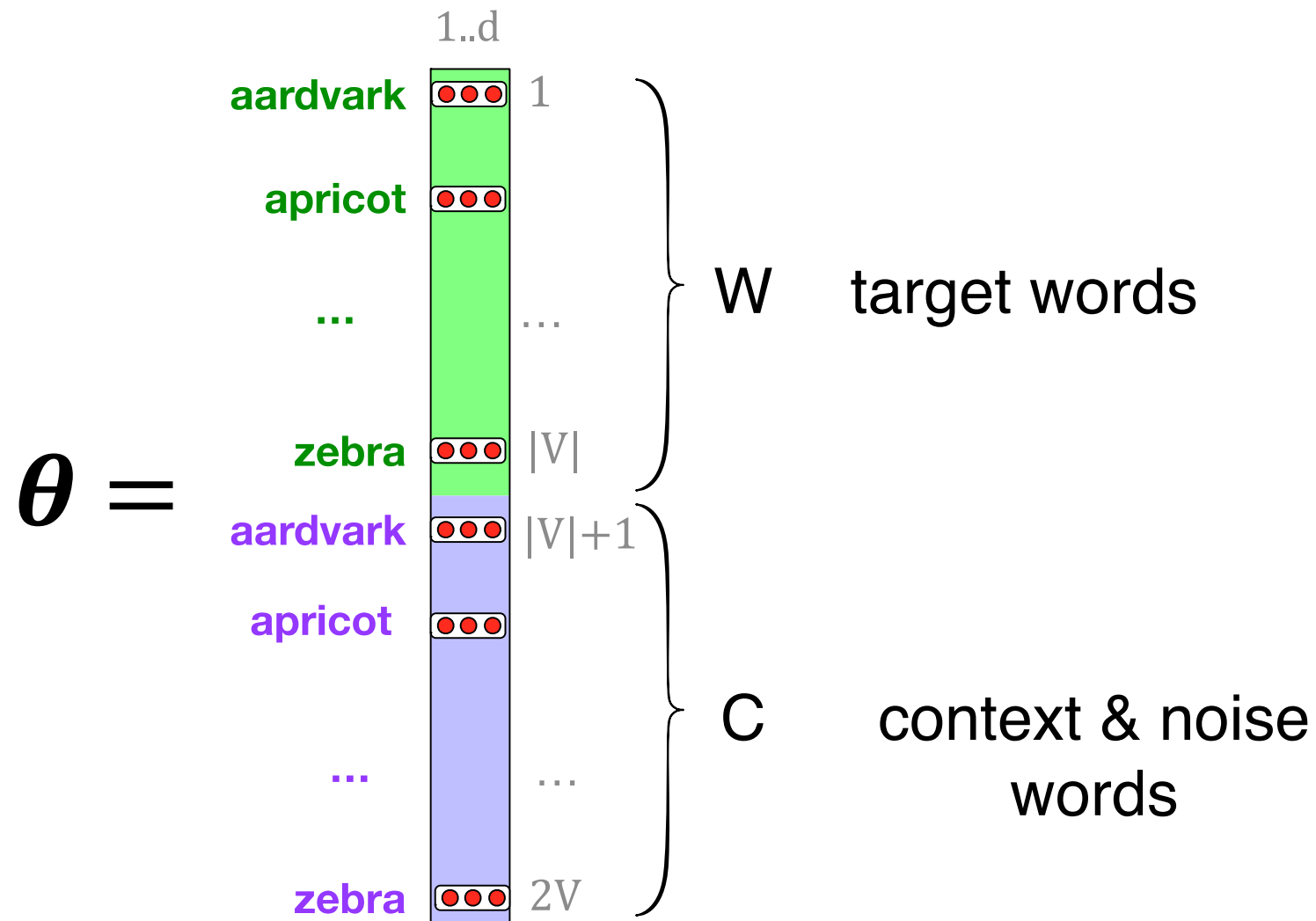
A probabilistic classifier, given

- a test target word w
- its context window of L words $c_{1:L}$

Estimates probability that w occurs in this window based on similarity of w (embeddings) to $c_{1:L}$ (embeddings).

To compute this, we just need embeddings for all the words.

These embeddings we'll need: a set for w , a set for c



Vector Semantics & Embeddings

Word2vec

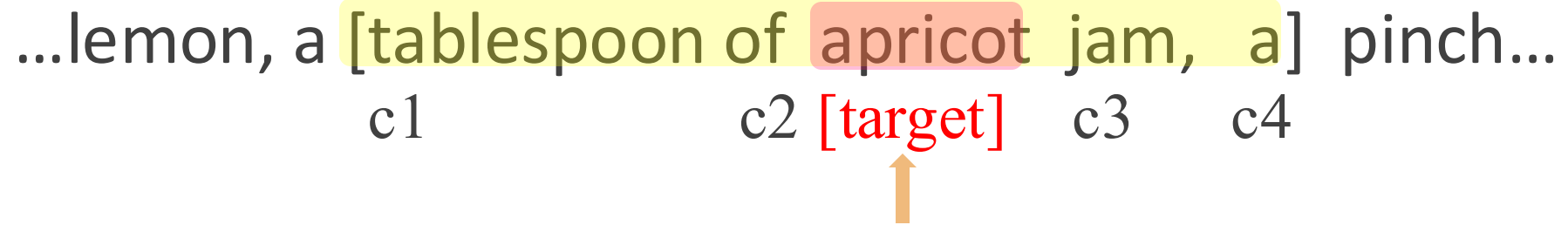
Vector Semantics & Embeddings

Word2vec: Learning the embeddings

Skip-Gram Training data

...lemon, a [tablespoon of apricot jam, a] pinch...

c1 c2 [target] c3 c4



positive examples +

t

c

apricot tablespoon

apricot of

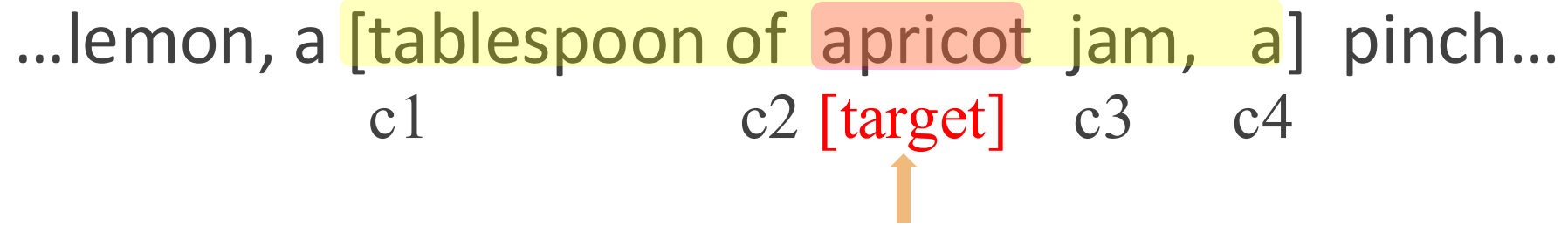
apricot jam

apricot a

Skip-Gram Training data

...lemon, a [tablespoon of apricot jam, a] pinch...

c1 c2 [target] c3 c4



positive examples +

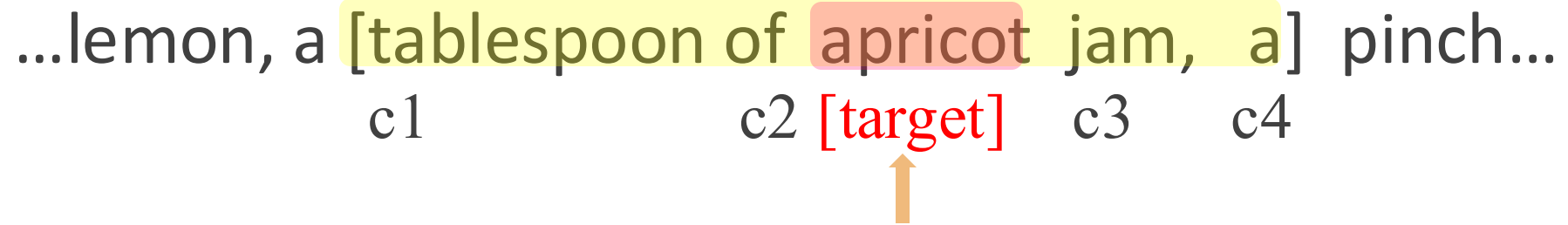
t	c
apricot	tablespoon
apricot	of
apricot	jam
apricot	a

For each positive example we'll grab k negative examples, sampling by frequency

Skip-Gram Training data

...lemon, a [tablespoon of apricot jam, a] pinch...

c1 c2 [target] c3 c4



positive examples +

t	c
apricot	tablespoon
apricot	of
apricot	jam
apricot	a

negative examples -

t	c	t	c
apricot	aardvark	apricot	seven
apricot	my	apricot	forever
apricot	where	apricot	dear
apricot	coaxial	apricot	if

Word2vec: how to learn vectors

Given the set of positive and negative training instances, and an initial set of embedding vectors

The goal of learning is to adjust those word vectors such that we:

- **Maximize** the similarity of the **target word, context word** pairs (w, c_{pos}) drawn from the positive data
- **Minimize** the similarity of the (w, c_{neg}) pairs drawn from the negative data.

Loss function for one w with c_{pos} , $c_{neg1} \dots c_{negk}$

Maximize the similarity of the target with the actual context words, and minimize the similarity of the target with the k negative sampled non-neighbor words.

$$\begin{aligned} L_{CE} &= -\log \left[P(+|w, c_{pos}) \prod_{i=1}^k P(-|w, c_{neg_i}) \right] \\ &= - \left[\log P(+|w, c_{pos}) + \sum_{i=1}^k \log P(-|w, c_{neg_i}) \right] \\ &= - \left[\log P(+|w, c_{pos}) + \sum_{i=1}^k \log (1 - P(+|w, c_{neg_i})) \right] \\ &= - \left[\log \sigma(c_{pos} \cdot w) + \sum_{i=1}^k \log \sigma(-c_{neg_i} \cdot w) \right] \end{aligned}$$

Learning the classifier

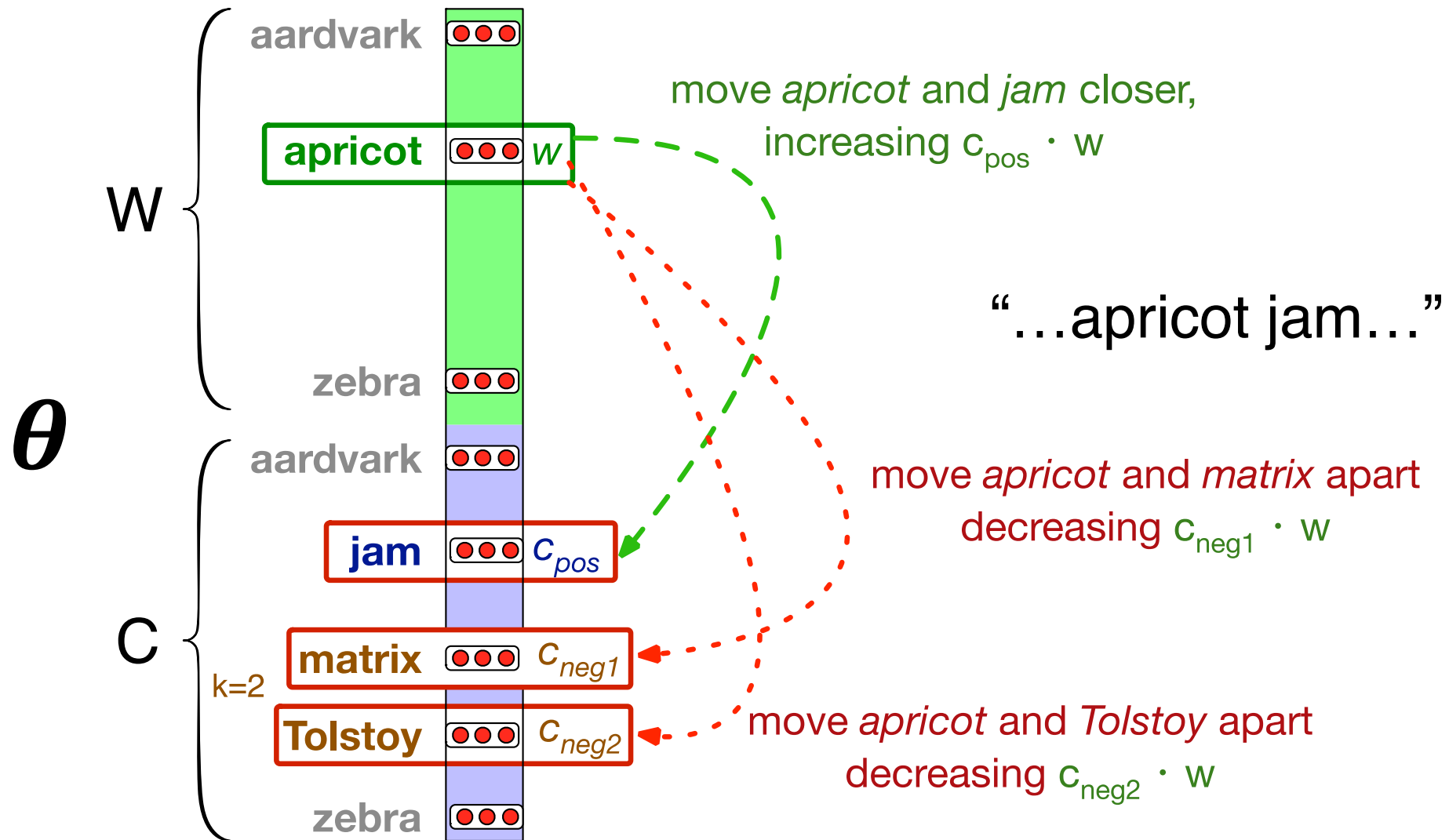
How to learn?

- Stochastic gradient descent!

We'll adjust the word weights to

- make the positive pairs more likely
- and the negative pairs less likely,
- over the entire training set.

Intuition of one step of gradient descent



Two sets of embeddings

SGNS learns two sets of embeddings

Target embeddings matrix W

Context embedding matrix C

It's common to just add them together,
representing word i as the vector $w_i + c_i$

Summary: How to learn word2vec (skip-gram) embeddings

Start with V random d -dimensional vectors as initial embeddings

Train a classifier based on embedding similarity

- Take a corpus and take pairs of words that co-occur as positive examples
- Take pairs of words that don't co-occur as negative examples
- Train the classifier to distinguish these by slowly adjusting all the embeddings to improve the classifier performance
- Throw away the classifier code and keep the embeddings.

Vector Semantics & Embeddings

Word2vec: Learning the embeddings

Vector Semantics & Embeddings

Properties of Embeddings

The kinds of neighbors depend on window size

Small windows ($C = \pm 2$) : nearest words are syntactically similar words in same taxonomy

- *Hogwarts* nearest neighbors are other fictional schools
- *Sunnydale, Evernight, Blandings*

Large windows ($C = \pm 5$) : nearest words are related words in same semantic field

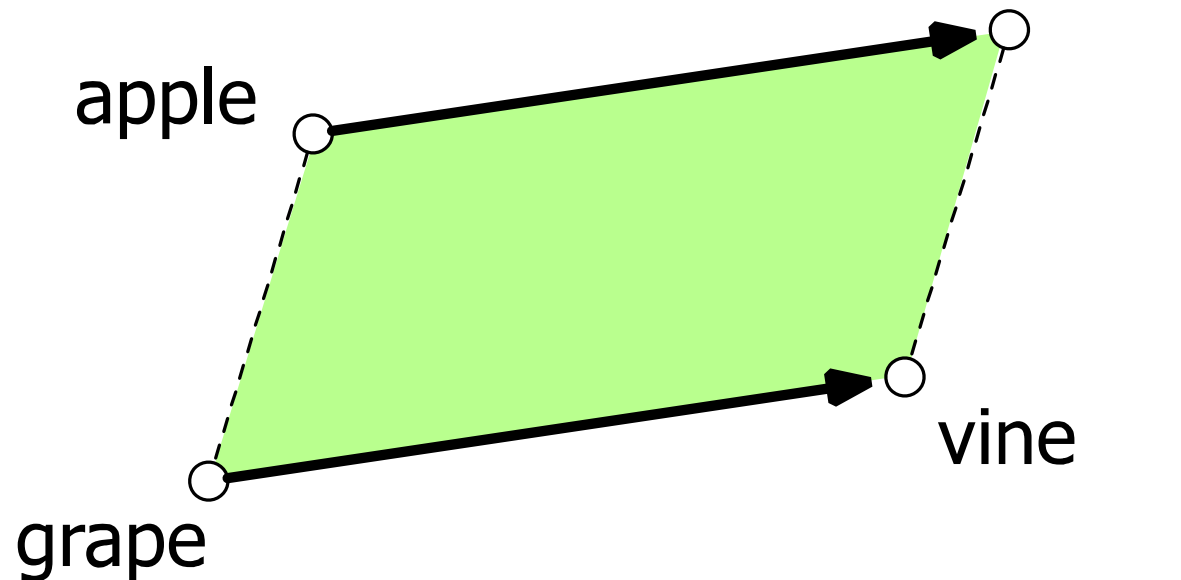
- *Hogwarts* nearest neighbors are Harry Potter world:
- *Dumbledore, half-blood, Malfoy*

Analogical relations

The classic parallelogram model of analogical reasoning
(Rumelhart and Abrahamson 1973)

To solve: *"apple is to tree as grape is to _____"*

Add $\overrightarrow{\text{tree} - \text{apple}}$ to $\overrightarrow{\text{grape}}$ to get ***vine***



Analogical relations via parallelogram

The parallelogram method can solve analogies with both sparse and dense embeddings (Turney and Littman 2005, Mikolov et al. 2013b)

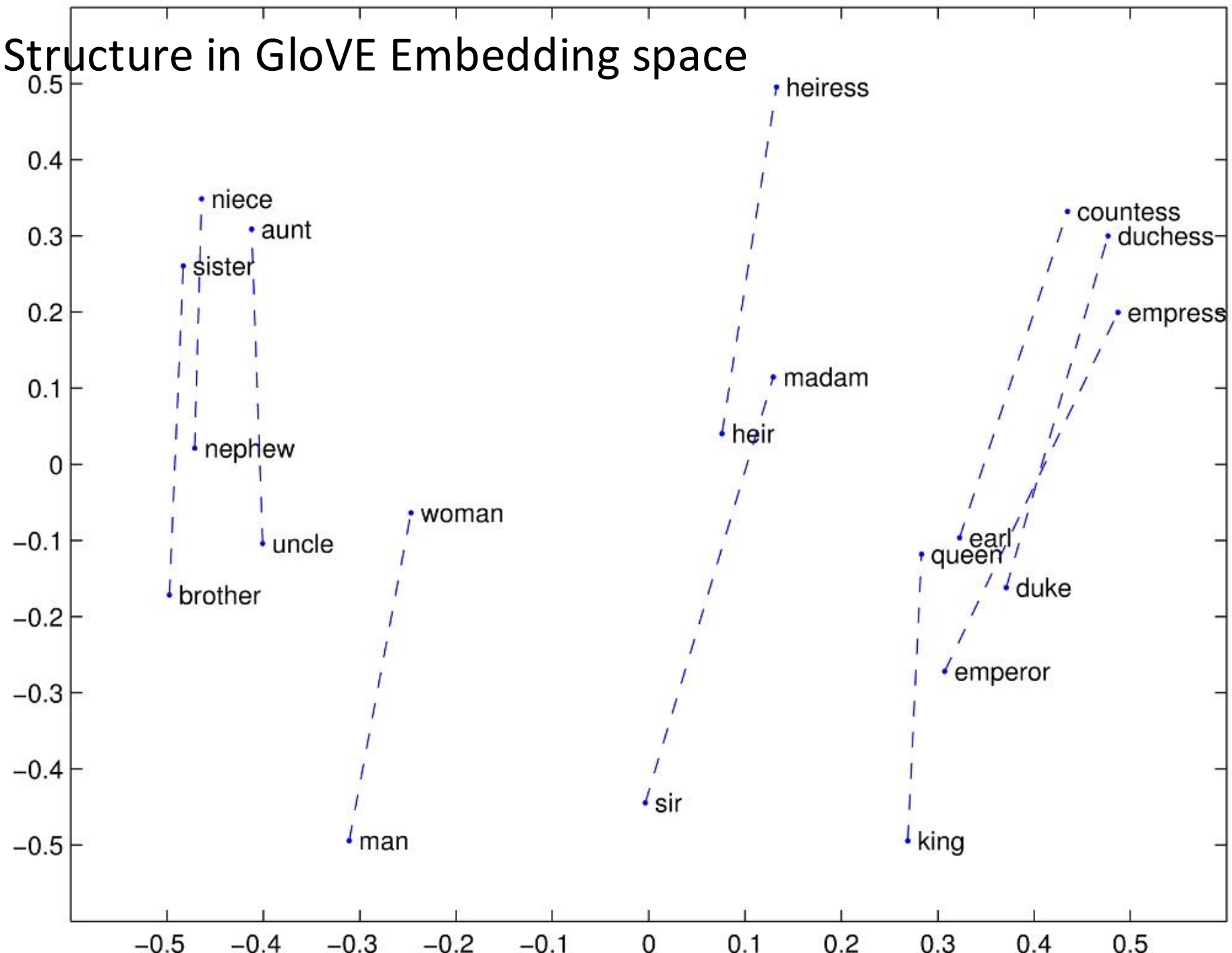
$\overrightarrow{\text{king}} - \overrightarrow{\text{man}} + \overrightarrow{\text{woman}}$ is close to $\overrightarrow{\text{queen}}$

$\overrightarrow{\text{Paris}} - \overrightarrow{\text{France}} + \overrightarrow{\text{Italy}}$ is close to $\overrightarrow{\text{Rome}}$

For a problem $a:a^*:b:b^*$, the parallelogram method is:

$$\hat{b}^* = \operatorname{argmax}_x \operatorname{distance}(x, a^* - a + b)$$

Structure in GloVe Embedding space



Caveats with the parallelogram method

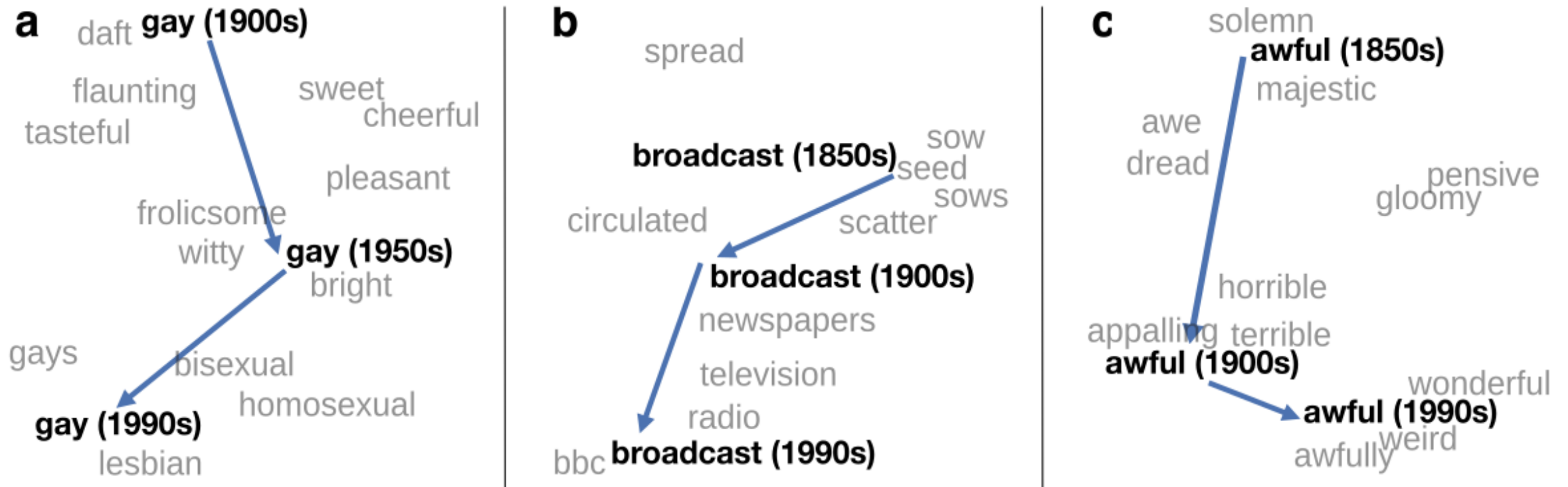
It only seems to work for frequent words, small distances and certain relations (relating countries to capitals, or parts of speech), but not others. (Linzen 2016, Gladkova et al. 2016, Ethayarajh et al. 2019a)

Understanding analogy is an open area of research (Peterson et al. 2020)

Embeddings as a window onto historical semantics

Train embeddings on different decades of historical text to see meanings shift

~30 million books, 1850-1990, Google Books data



William L. Hamilton, Jure Leskovec, and Dan Jurafsky. 2016. Diachronic Word Embeddings Reveal Statistical Laws of Semantic Change. Proceedings of ACL.

Embeddings reflect cultural bias!

Bolukbasi, Tolga, Kai-Wei Chang, James Y. Zou, Venkatesh Saligrama, and Adam T. Kalai. "Man is to computer programmer as woman is to homemaker? debiasing word embeddings." In *NeurIPS*, pp. 4349-4357. 2016.

Ask “Paris : France :: Tokyo : x”

- x = Japan

Ask “father : doctor :: mother : x”

- x = nurse

Ask “man : computer programmer :: woman : x”

- x = homemaker

Algorithms that use embeddings as part of e.g., hiring searches for programmers, might lead to bias in hiring

Historical embedding as a tool to study cultural biases

Garg, N., Schiebinger, L., Jurafsky, D., and Zou, J. (2018). Word embeddings quantify 100 years of gender and ethnic stereotypes. *Proceedings of the National Academy of Sciences* 115(16), E3635–E3644.

- Compute a **gender or ethnic bias** for each adjective: e.g., how much closer the adjective is to "woman" synonyms than "man" synonyms, or names of particular ethnicities
 - Embeddings for **competence** adjective (*smart, wise, brilliant, resourceful, thoughtful, logical*) are biased toward men, a bias slowly decreasing 1960-1990
 - Embeddings for **dehumanizing** adjectives (barbaric, monstrous, bizarre) were biased toward Asians in the 1930s, bias decreasing over the 20th century.
- These match the results of old surveys done in the 1930s

Vector Semantics & Embeddings

Properties of Embeddings

Contextual Word Embeddings



Borrowed from:

COS 598C Advanced Topics in Computer Science: Deep
Learning for Natural Language Processing

Overview

- (McCann et al, NIPS'2017) **Learned in translation: contextualized word vectors**
- (Peters et al, NAACL'2018) **Deep contextualized word representations**

Limitations of word2vec

- One vector for each word type

$$v(\text{bank}) = \begin{pmatrix} -0.224 \\ 0.130 \\ -0.290 \\ 0.276 \end{pmatrix}$$

- Polysemous words, e.g., bank, mouse

mouse¹ : a *mouse* controlling a computer system in 1968.

mouse² : a quiet animal like a *mouse*

bank¹ : ...a *bank* can hold the investments in a custodial account ...

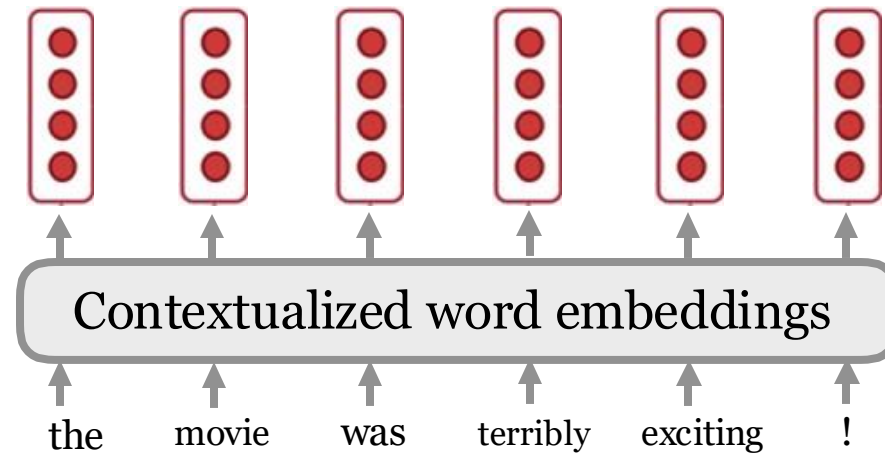
bank² : ...as agriculture burgeons on the east *bank*, the river ...

- Words don't appear in isolation. The word use (e.g., syntax and semantics) depends on its context. Why not learn the representations for each word in its context?

Contextualized word embeddings

Build a vector for each word conditioned on its **context**!

= representation for each token is a function of the entire input sentence



$$g : (w_1, w_2, \dots, w_n) \longrightarrow \mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$$

Contextualized word embeddings

Compute contextual vector:

$$\mathbf{c}_k = f(w_k \mid w_1, w_2, \dots, w_n) \in \mathbb{R}^d$$

$f(\text{play} \mid \text{Elmo and Cookie Monster play a game})$

⊗

$f(\text{play} \mid \text{The Broadway play premiered yesterday})$

Contextualized word embeddings

	Source	Nearest Neighbors
GloVe	play	playing, game, games, played, players, plays, player, Play, football, multiplayer
biLM	Chico Ruiz made a spectacular <u>play</u> on Alusik 's grounder {...}	Kieffer , the only junior in the group , was commended for his ability to hit in the clutch , as well as his all-round excellent <u>play</u> .
	Olivia De Havilland signed to do a Broadway <u>play</u> for Garson {...}	{...} they were actors who had been handed fat roles in a successful <u>play</u> , and had talent enough to fill the roles competently , with nice understatement .

(Peters et al, 2018): Deep contextualized word representations

Noah Smith. 2019.

Contextual Word Representations: A Contextual Introduction

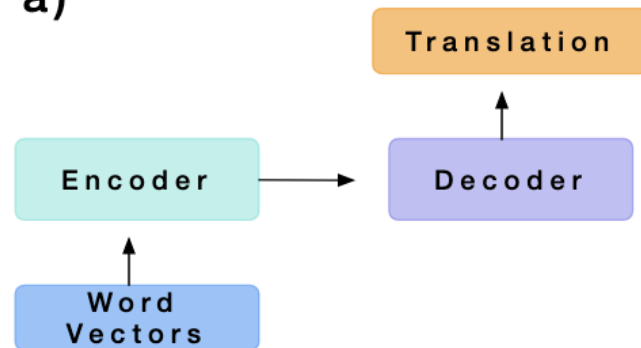
“With hindsight, we can now see that by representing word types independent of context, we were solving a problem that was harder than it needed to be. Because words mean different things in different contexts, we were requiring that type representations capture all of the possibilities. Moving to word token vectors simplifies things, asking the word token representation to capture only what a word means in this context. For the same reasons that the collection of contexts a word type is found in provide clues about its meaning(s), a particular token’s context provides clues about its specific meaning.”

CoVe

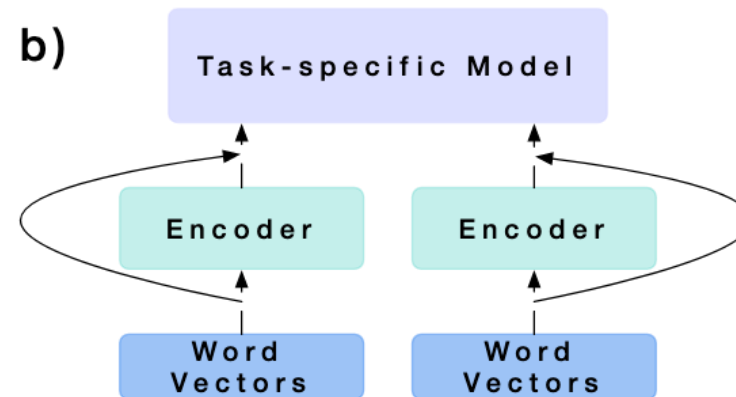
Key idea:

- Train a standard sequence-to-sequence (with attention) model for English-to-German translation.
- Take the encoder directly as contextualized word embeddings.

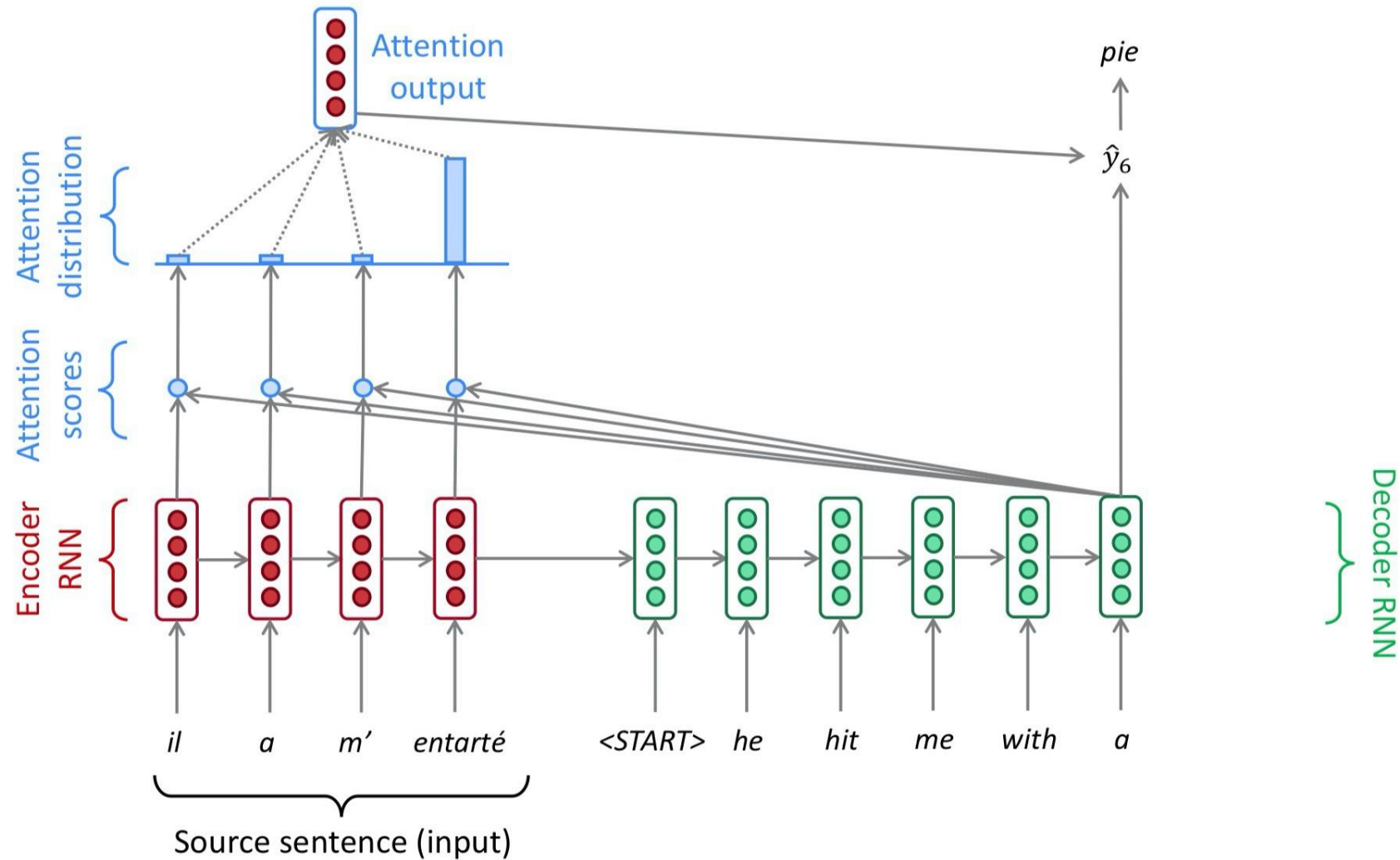
a)



b)



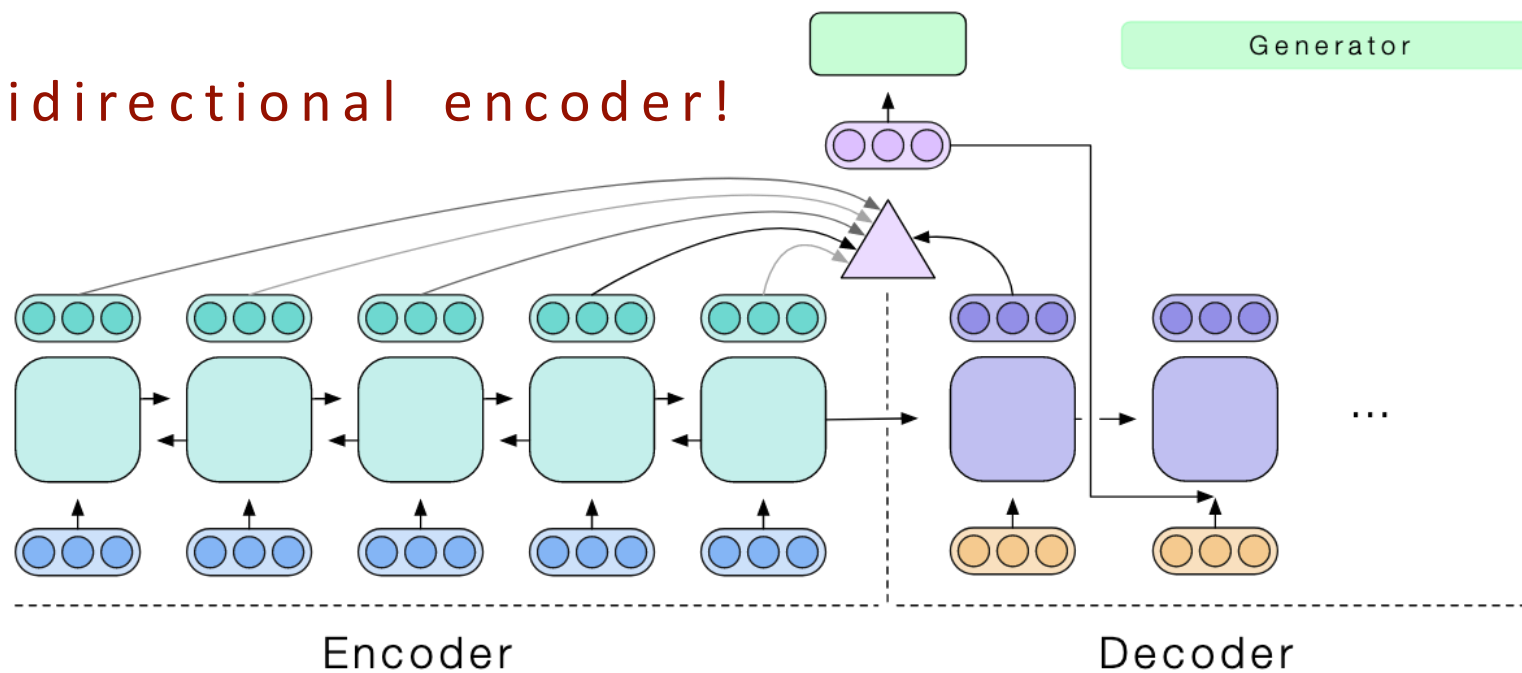
Sequence-to-sequence with attention



(slide credit: Abigail See)

CoVe

Bidirectional encoder!



$$\text{CoVe}(w) = \text{MT-LSTM}(\text{GloVe}(w))$$

$$\tilde{w} = [\text{GloVe}(w); \text{CoVe}(w)]$$

Training details

Machine Translation datasets

- Small: 30,000 sentence pairs (Flickr captions)
- Medium: 209,772 (TED presentations)
- Large: 7M (Web crawl data, news, European Parliament proceedings...)

- Encoder: two-layer **bidirectional** LSTMs
- Decoder: two-layer unidirectional LSTMs

Evaluation

Dataset	Task	Details	Examples
SST-2	Sentiment Classification	2 classes, single sentences	56.4k
SST-5	Sentiment Classification	5 classes, single sentences	94.2k
IMDb	Sentiment Classification	2 classes, multiple sentences	22.5k
TREC-6	Question Classification	6 classes	5k
TREC-50	Question Classification	50 classes	5k
SNLI	Entailment Classification	2 classes	550k
SQuAD	Question Answering	open-ended (answer-spans)	87.6k

Evaluation: SNLI

Text	Judgments	Hypothesis
A man inspects the uniform of a figure in some East Asian country.	contradiction C C C C C	The man is sleeping
An older and younger man smiling.	neutral N N E N N	Two men are smiling and laughing at the cats playing on the floor.
A black race car starts up in front of a crowd of people.	contradiction C C C C C	A man is driving down a lonely road.
A soccer game with multiple males playing.	entailment E E E E E	Some men are playing a sport.
A smiling costumed woman is holding an umbrella.	neutral N N E C N	A happy woman in a fairy costume holds an umbrella.

Evaluation: SQuAD

Passage

Super Bowl 50 was an American football game to determine the champion of the National Football League (NFL) for the 2015 season. The American Football Conference (AFC) champion Denver Broncos defeated the National Football Conference (NFC) champion Carolina Panthers 24–10 to earn their third Super Bowl title. The game was played on February 7, 2016, at Levi's Stadium in the San Francisco Bay Area at Santa Clara, California.

Question: Which NFL team won Super Bowl 50?

Answer: Denver Broncos

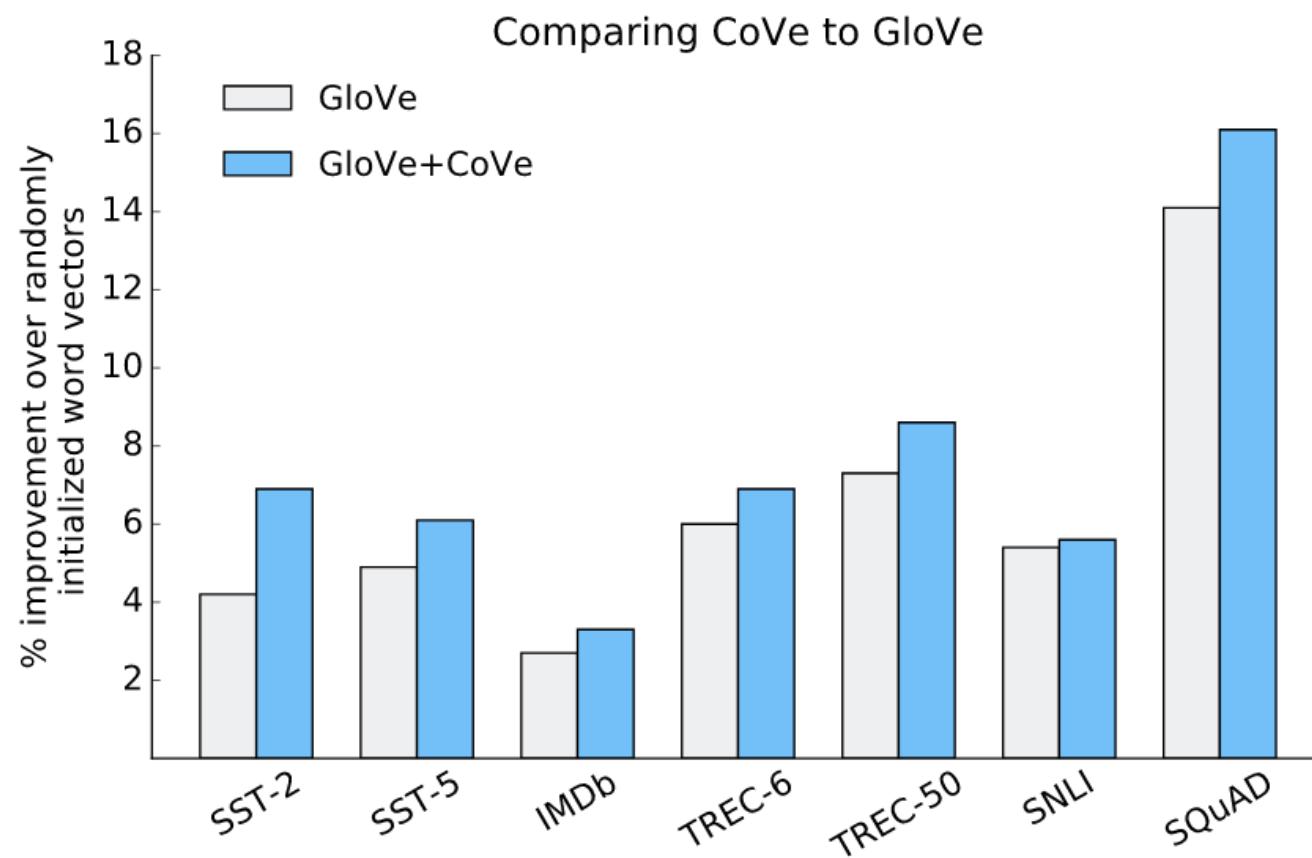
Question: What does AFC stand for?

Answer: American Football Conference

Question: What year was Super Bowl 50?

Answer: 2016

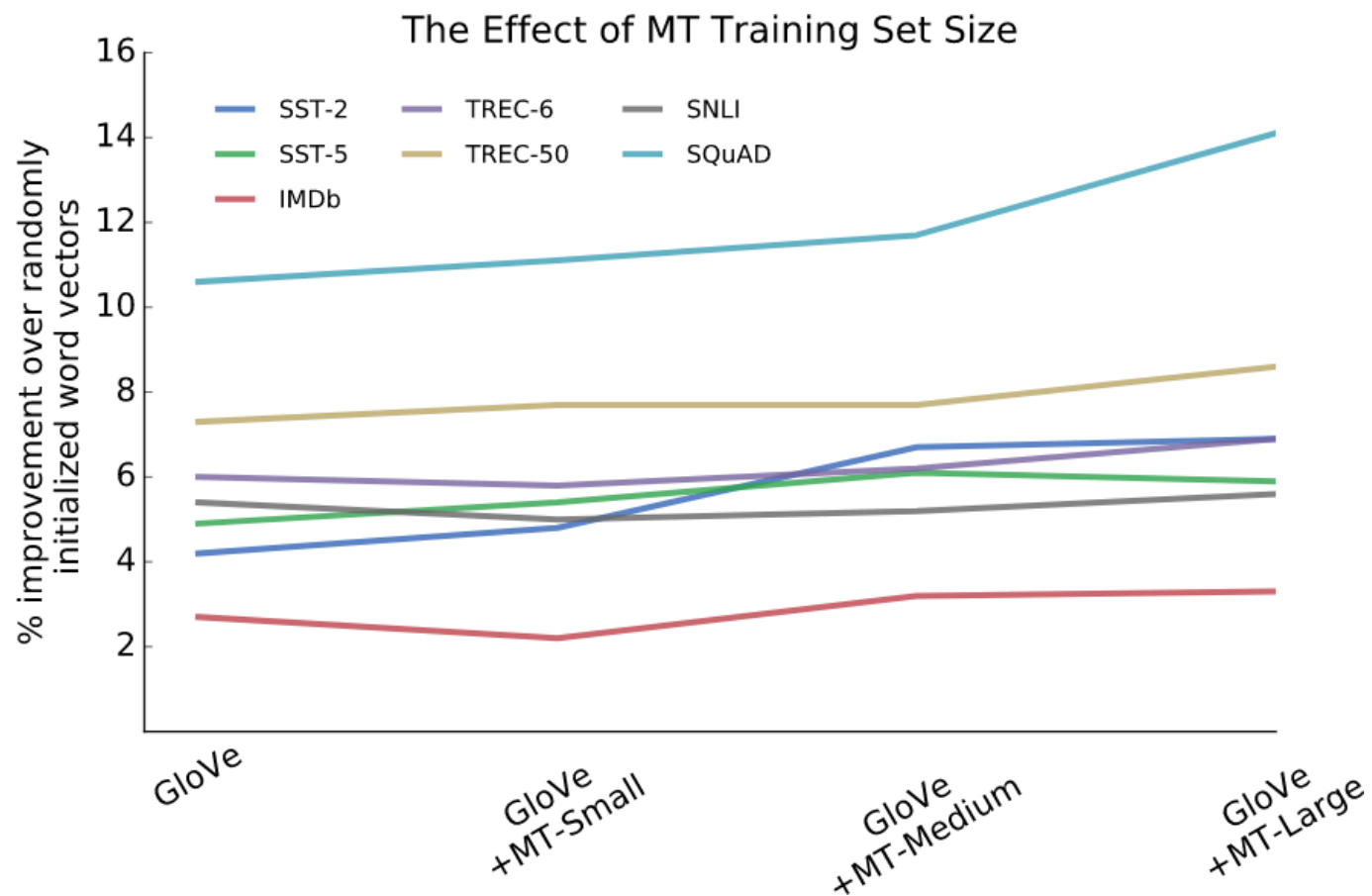
Evaluation



Evaluation

Dataset	Random	GloVe	GloVe+				
			Char	CoVe-S	CoVe-M	CoVe-L	Char+CoVe-L
SST-2	84.2	88.4	90.1	89.0	90.9	91.1	91.2
SST-5	48.6	53.5	52.2	54.0	54.7	54.5	55.2
IMDb	88.4	91.1	91.3	90.6	91.6	91.7	92.1
TREC-6	88.9	94.9	94.7	94.7	95.1	95.8	95.8
TREC-50	81.9	89.2	89.8	89.6	89.6	90.5	91.2
SNLI	82.3	87.7	87.7	87.3	87.5	87.9	88.1
SQuAD	65.4	76.0	78.1	76.5	77.1	79.5	79.9

More MT —> Better CoVe



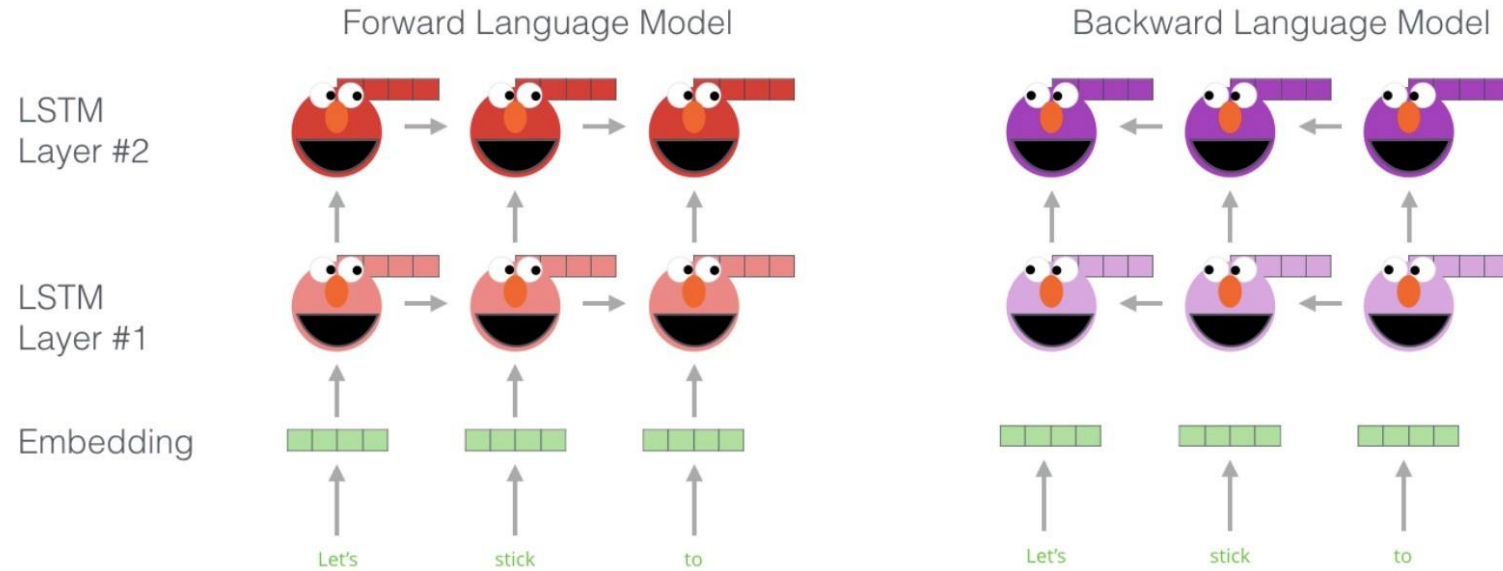
ELMo

Key idea:

- Train a forward LSTM-based LM and a backward LSTM-based LM on some large corpus
- Use the hidden states of the LSTMs for each token to compute a vector representation of each word



ELMo



words in the sentence $\rightarrow$

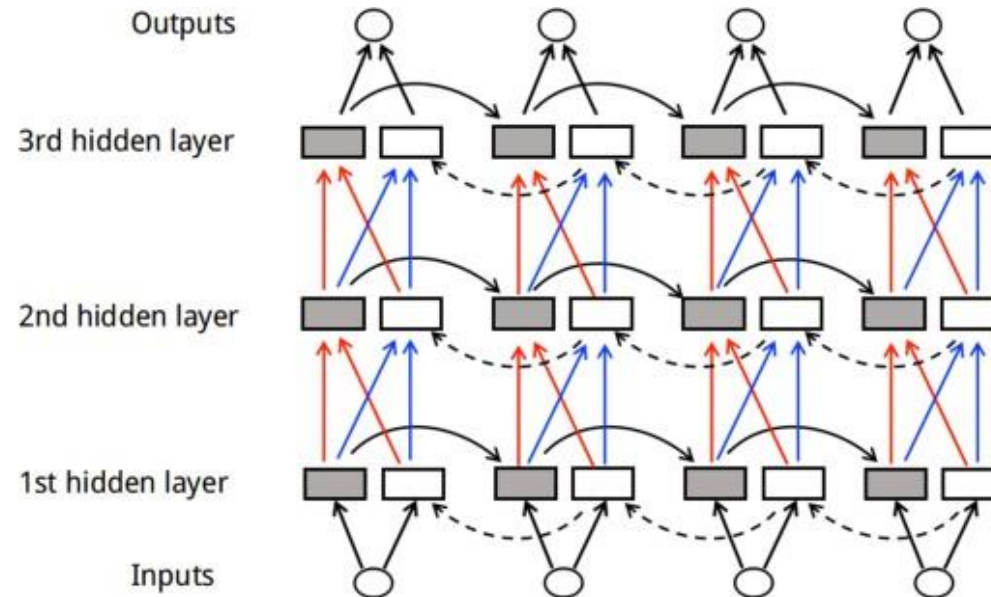
$$\sum_{k=1}^N \left(\log p(t_k \mid t_1, \dots, t_{k-1}; \Theta_x, \vec{\Theta}_{LSTM}, \Theta_s) \right. \\ \left. + \log p(t_k \mid t_{k+1}, \dots, t_N; \Theta_x, \overleftarrow{\Theta}_{LSTM}, \Theta_s) \right)$$

input

softmax

ELMo!=bidirectional LSTMs

- Essentially two LSTMs in different directions



How to use ELMo?

$$\begin{aligned} R_k &= \{\mathbf{x}_k^{LM}, \vec{\mathbf{h}}_{k,j}^{LM}, \overleftarrow{\mathbf{h}}_{k,j}^{LM} \mid j = 1, \dots, L\} \\ &= \{\mathbf{h}_{k,j}^{LM} \mid j = 0, \dots, L\}, \end{aligned}$$

of layers 

$$\mathbf{h}_{k,0}^{LM} = \mathbf{x}_k^{LM}, \mathbf{h}_{k,j}^{LM} = [\vec{\mathbf{h}}_{k,j}^{LM}; \overleftarrow{\mathbf{h}}_{k,j}^{LM}]$$

$$\mathbf{ELMo}_k^{task} = E(R_k; \Theta^{task}) = \gamma^{task} \sum_{j=0}^L s_j^{task} \mathbf{h}_{k,j}^{LM}$$

- γ^{task} : allows the task model to scale the entire ELMo vector
- s_j^{task} : softmax-normalized weights across layers

How to use ELMo?

- Plug ELMo into any (neural) NLP model: freeze all the LMs weights and change the input representation to:

$$[\mathbf{x}_k; \mathbf{ELMo}_k^{task}]$$

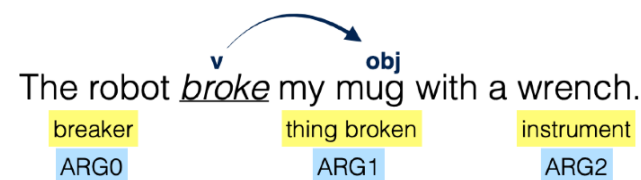
- Could also insert ELMo into higher layers (will discuss soon)

Training details

- Forward and backward LMs: 2 layers each
- Character embeddings to replace word embeddings
 - *Recommended reading: Character-Aware Neural Language Models*
 - 2048 char n-gram filters and 2 highway layers, 512 dim projection
No-pretrained word embeddings!
- User 4096 dim hidden/cell LSTM states with 512 dim projections to next input
- A residual connection from the first to second layer
- Trained 10 epochs on 1B Word Benchmark

Evaluation

- SQuAD: question answering ✓
- SNLI: textual entailment ✓
- SRL: semantic role labeling
- Coref: coreference resolution
- NER: named entity recognition
- SST-5: sentiment analysis ✓

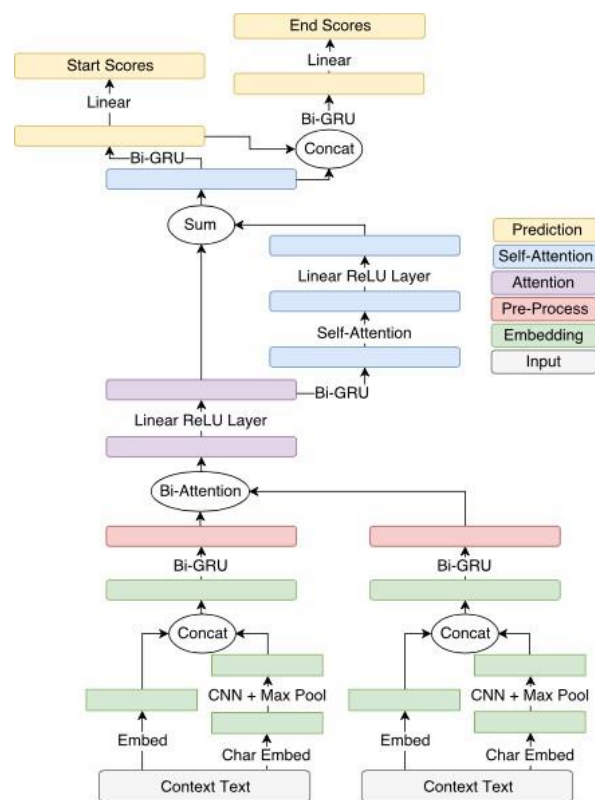


Barack Obama nominated Hillary Rodham Clinton as his secretary of state on Monday. He chose her because she had foreign affairs experience as a former First Lady.

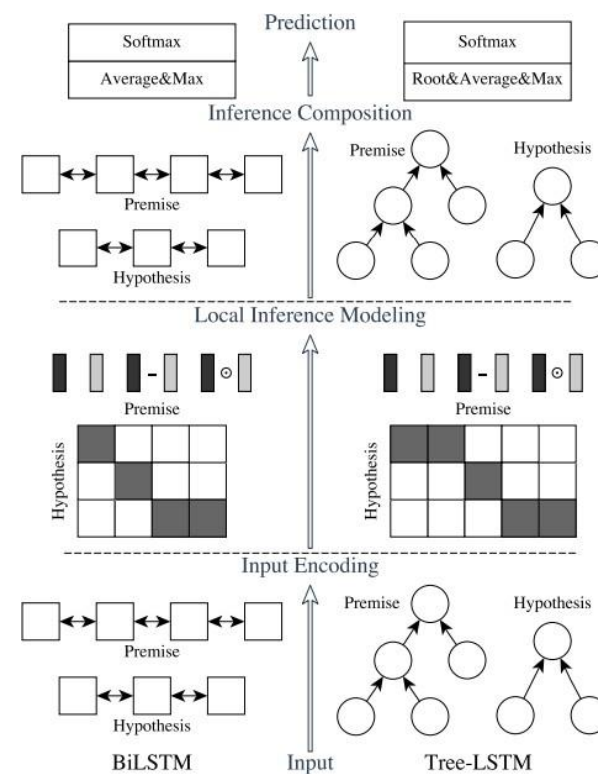
Baseline models

- SQuAD: (Clark and Gardner, 2017)
- Textual entailment: (Chen et al, 2017)
- Semantic role labeling: (He et al, 2017)
- Coreference resolution: (Lee et al, 2017)
- Named entity recognition: pre-trained word embeddings, a character-based CNN representation, two biLSTM layers and a conditional random field (CRF) loss
- Sentiment analysis: (McCann et al, 2017)

Baseline models



(Clark and Gardner, 2017)



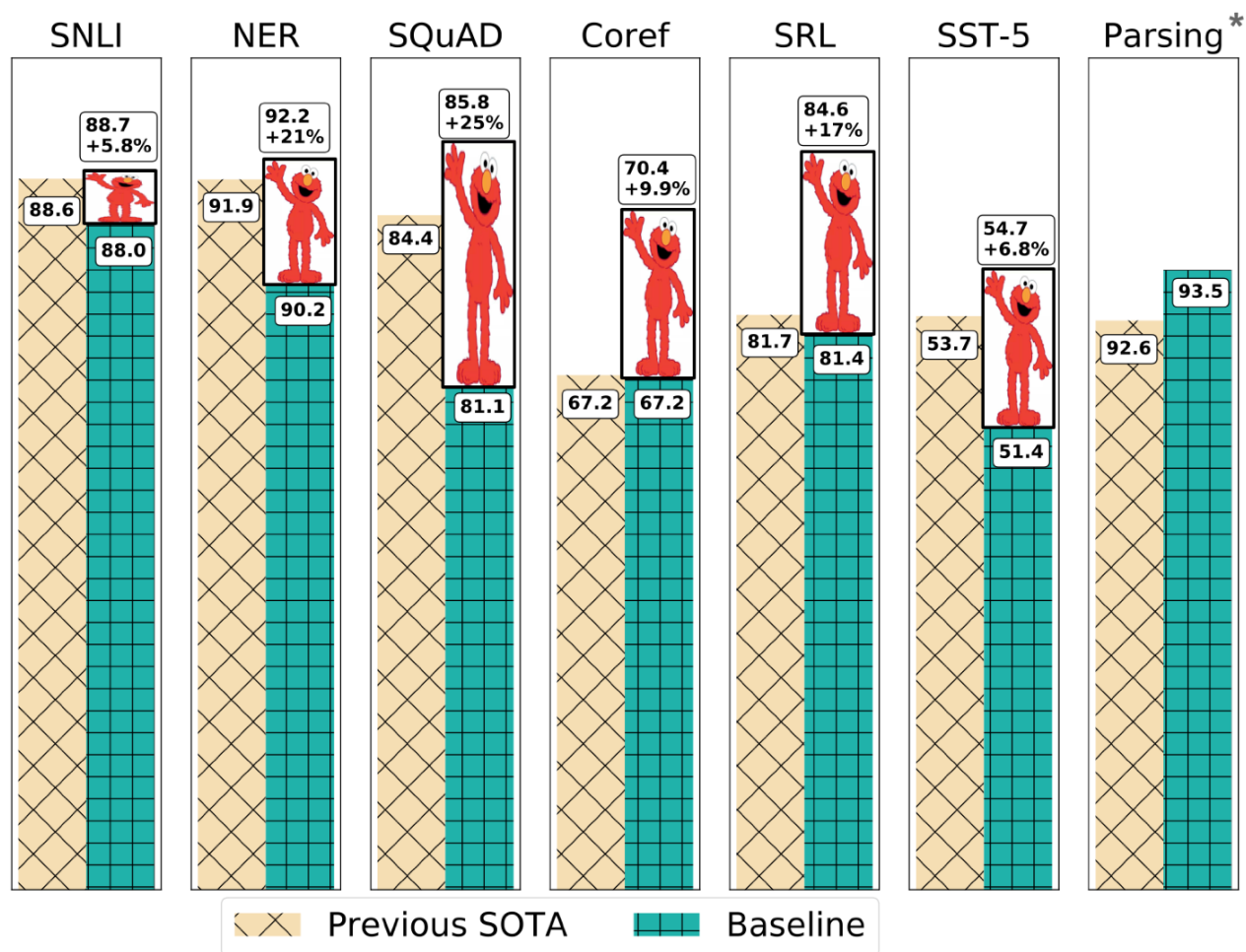
(Chen et al, 2017)

Experimental results

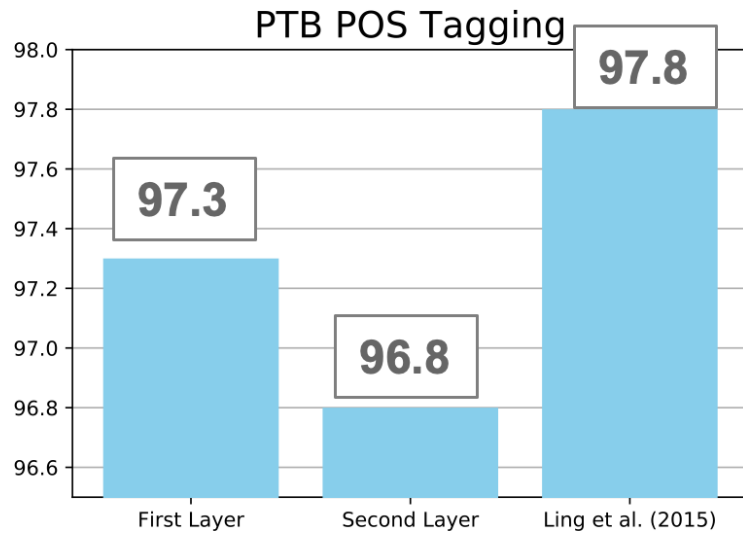
TASK	PREVIOUS SOTA		OUR BASELINE	ELMo + BASELINE	INCREASE (ABSOLUTE/ RELATIVE)
SQuAD	Liu et al. (2017)	84.4	81.1	85.8	4.7 / 24.9%
SNLI	Chen et al. (2017)	88.6	88.0	88.7 ± 0.17	0.7 / 5.8%
SRL	He et al. (2017)	81.7	81.4	84.6	3.2 / 17.2%
Coref	Lee et al. (2017)	67.2	67.2	70.4	3.2 / 9.8%
NER	Peters et al. (2017)	91.93 ± 0.19	90.15	92.22 ± 0.10	2.06 / 21%
SST-5	McCann et al. (2017)	53.7	51.4	54.7 ± 0.5	3.3 / 6.8%

- SQuAD: “The increase of 4.7% with ELMo is also significantly larger than the 1.8% improvement from adding CoVe to a baseline model”
- SST-5: “Replacing CoVe with ELMo results in a 1.0% absolute accuracy”

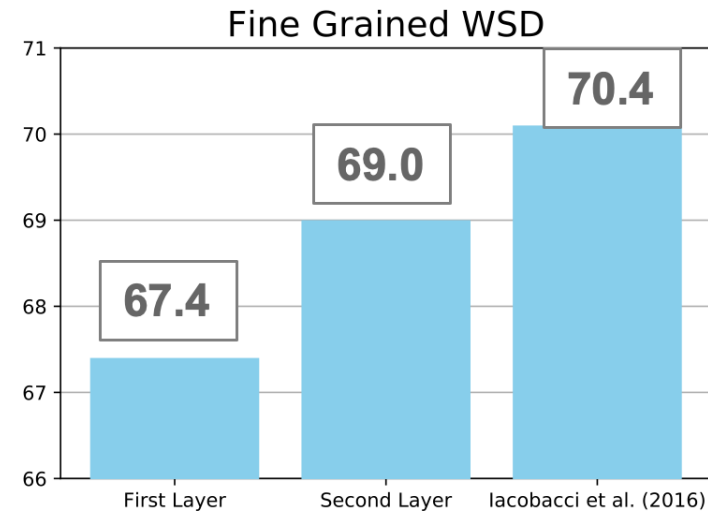
Experimental results



What information is captured by ELMo representations?



First Layer > Second Layer



Second Layer > First Layer

syntactic information is better represented at lower layers
while semantic information is captured at higher layers

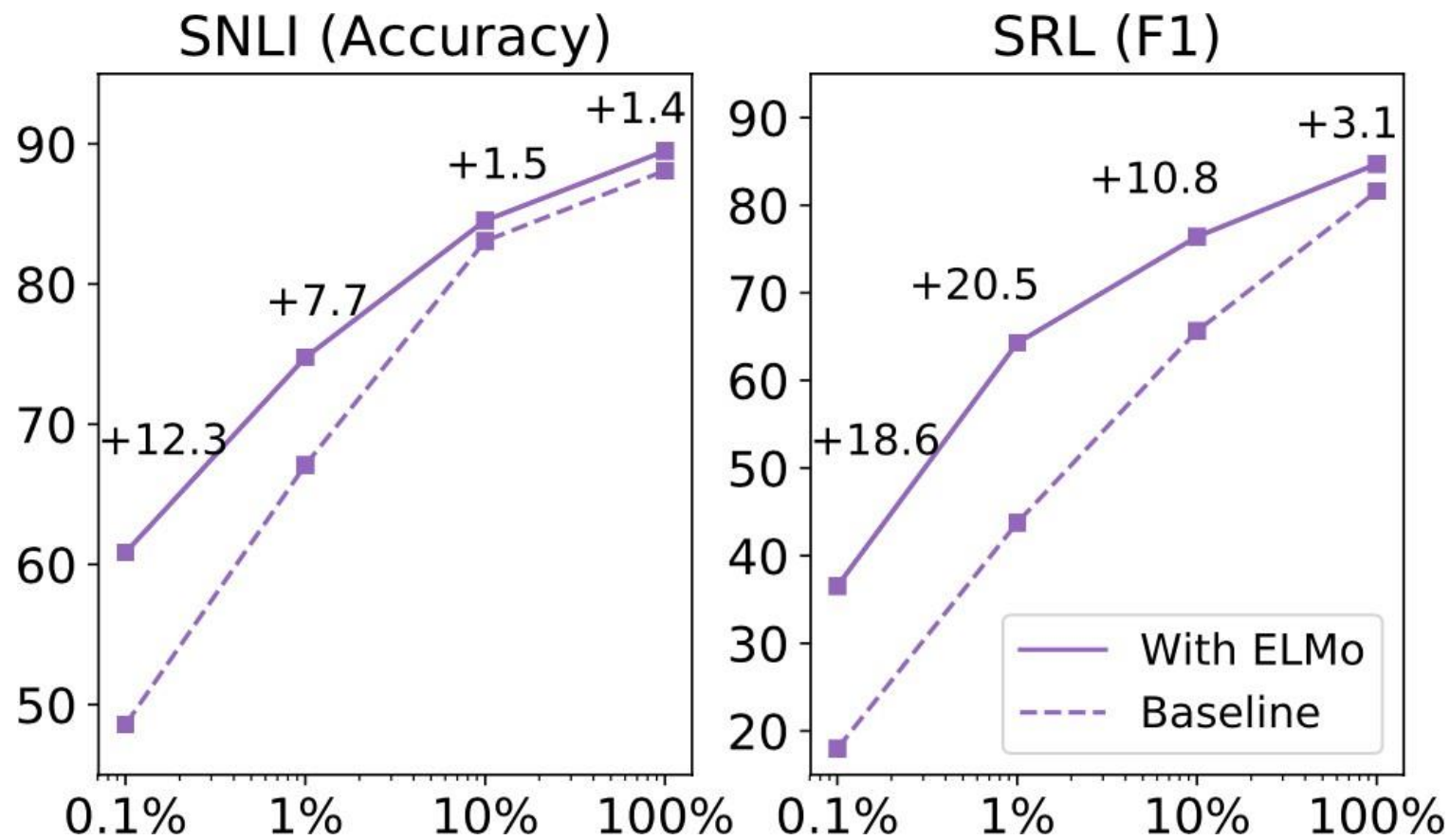
Where to include ELMo?

Task	Input Only	Input & Output	Output Only
SQuAD	85.1	85.6	84.8
SNLI	88.9	89.5	88.7
SRL	84.7	84.3	80.9

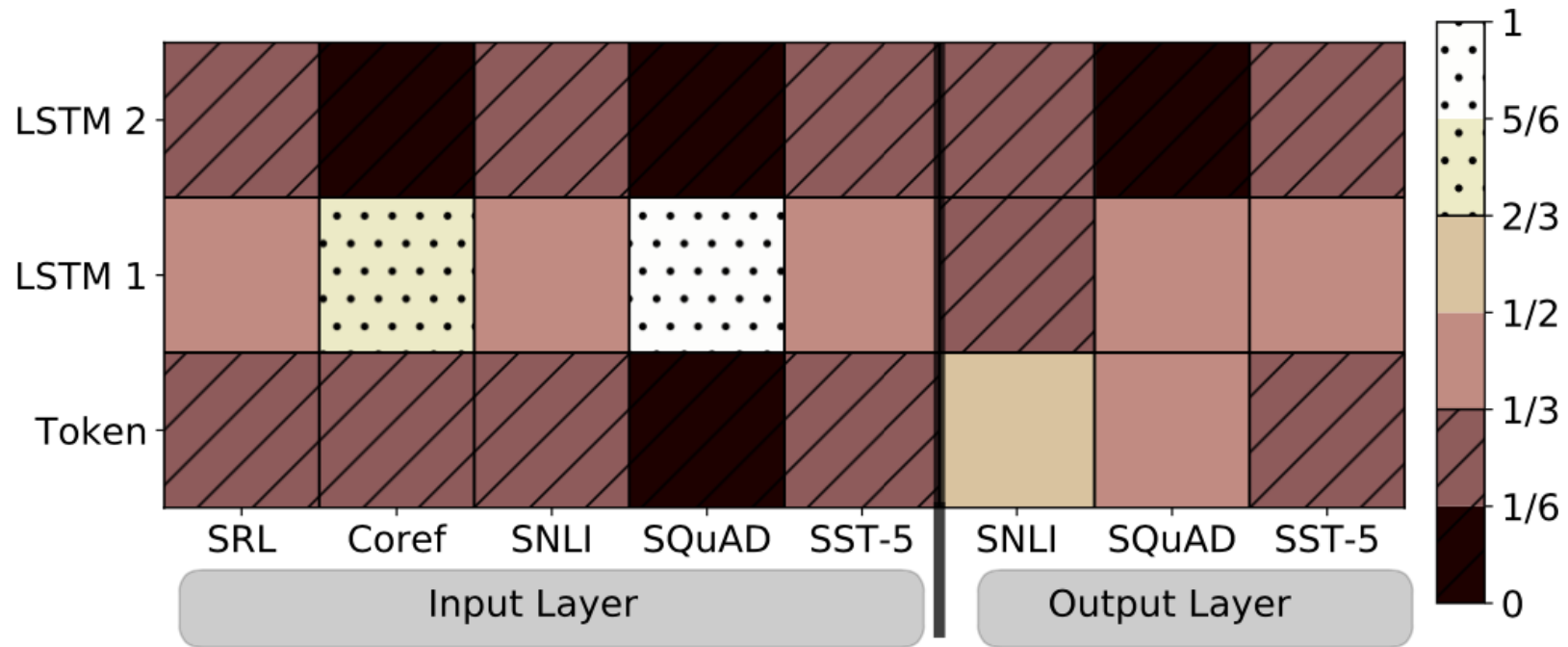
Using all layers > last layer

Task	Baseline	Last Only	Reg. Parameter	
			All layers	
			$\lambda=1$	$\lambda=0.001$
SQuAD	80.8	84.7	85.0	85.2
SNLI	88.1	89.1	89.3	89.5
SRL	81.6	84.1	84.6	84.8

Sample Efficiency



Visualization of softmax normalized weights



Discussion: CoVe vs ELMo

- Data dependence: monolingual data vs parallel data
- Training efficiency: ELMo slightly better than CoVe
- Objectives: predicting each word in the sentence vs “translating source sentence as a whole”
- Directionality: bidirectional encoder vs forward + backward LMs

Limitations of ELMo/CoVe

- Task-specific architectures: Contextualized word embeddings are used as an augmentation to static word embeddings
- Trained on single sentences
- Training corpus is much smaller than those used for training word2vec/GloVe vectors